sold_out_concert_ids), which triggers our ActionCable broadcast with a hash of data that is identical to the data we would also send out as a normal render. Again, that's just sample coding here. Normally the ActionCable broadcast would not always be in the same place as the regular render command.

By default, the subscription sends the data right down to the client without further processing. If you'd like further processing, you can do a couple of things when you declare the stream in the channel using stream_from. The stream_from method takes an optional keyword argument that can have one value: coder: ActiveSupport::JSON. With that argument set, all incoming messages are decoded as JSON before passing on. Alternately, you can pass a block to stream_from; the messages are passed to the block and the result of the block is what is sent to the client.

Now we need to catch the data on our client side. From our previous example, we have our class SoldOutDataController that was polling the server to get new data about the list of sold-out concerts.

We just need to repurpose that class to set up an ActionCable subscription:

chapter_09/03/app/javascript/controllers/sold_out_data_controller.ts

```
import { Controller } from "@hotwired/stimulus"
import { createConsumer, Subscription } from "@rails/actioncable"

export default class SoldOutDataController extends Controller {
  static targets = ["concert"]
  concertTargets: Array<HTMLElement>
  subscription: Subscription
  started: boolean

  connect(): void {
    if (this.subscription) {
      return
    }
    this.started = true
    this.subscription = this.createSubscription(this)
  }
```